SPLUNK AGENTIC OPS HACKATHON · PLATFORM & DEVELOPER EXPERIENCE

DualContext

Production incident. Two contexts. One answer.
45 seconds, not 13 minutes.

Splunk MCP · SigMap MCP · Hosted Models · Groundedness-scored

Manoj Mallick · LearnHubPlay BV

THE PROBLEM

Incidents need two kinds of context

When production breaks, a developer needs operational reality **and** code structure — and today they live in two different tools.

Splunk

knows the symptom

47 errors, line 47, 8× baseline — but not *which file or method* owns it.

IDE

knows the code

The class and method — but not *which error* is firing in production right now.

13 min

tab-switching

Average investigation, alt-tabbing between dashboard and editor. Mental overhead.

Neither source alone is sufficient. Together they pinpoint both.

THE SOLUTION

One query, two MCP servers

DualContext fans one developer question out to two MCP servers in parallel, then a Splunk hosted model fuses the answer.

ASK one NL question → **SPLUNK MCP** errors · metrics · alerts + **SIGMAP MCP** ranked files · methods → **SYNTHESIZE** hosted model → **JUDGE** groundedness 0–1

Single-source tools stop here

“Here are your logs.” Or: “here’s your code.” The developer still has to bridge the two by hand.

DualContext bridges them

Cites a specific log *and* a specific file+method in one grounded answer — scored for groundedness every time.

HOW WE USE SPLUNK

The MCP Server is the protagonist

CAPABILITY	HOW DUALCONTEXT USES IT	PRIZE
MCP Server	Primary integration — operational context via `run_splunk_query` (handshake, Bearer auth)	Best Use of MCP
Hosted Models	`gpt-oss` synthesizes both contexts into one grounded answer	Best Use of Models
SPL	Errors, error-rate timechart, alert history — three searches per query	Core
SigMap MCP	Second MCP server — ranked code structure at ~97% token reduction	Composability

MCP as an **interop layer** — exactly what Splunk is promoting. Splunk does the heavy lifting; SigMap adds the code lens.

PROOF — RUNNABLE, SCORED

Grounded answer in ~1 second

Two MCP servers in parallel, fused and judged — query: *“Auth service throwing 47 errors per hour. What’s causing it?”*

```
SPLUNK MCP · 3 SPL searches, 47 events (1061 ms)
SIGMAP MCP · ranked 3 files, 180 tokens vs 45000 raw (99.6% reduction)
PARALLEL · both ready in 1061 ms (not 1862 ms serial)
SYNTHESIS · gpt-oss-120b fused both sources
JUDGE · groundedness 0.83 PASS (threshold 0.7)
```

Answer cites `JwtTokenProvider.validateToken()` (line 47) *and* the Splunk error spike — both, in one paragraph.

THE GROUNDEDNESS GUARANTEE

Every answer is scored

A groundedness judge scores how well the answer cites the actual files/methods — no silent hallucination.

WITH BOTH CONTEXTS	GENERIC ANSWER
0.83	**0.15**
Cites the specific log **and** the specific file+method. PASS.	“Spring Security typically uses…” — no codebase grounding. FAIL.

The developer sees the score with **every** response. Trust, made visible.

ARCHITECTURE

One query · dual MCP · fused answer

Developer question — natural language
"Auth service throwing 47 errors per hour. What's causing it?"

▼

Splunk MCP Server
run_splunk_query · errors · metrics · alerts

SigMap MCP Server
query_context · ranked files · 99.6% fewer tokens

▼

DualContext Agent

PARALLEL	both MCP servers queried concurrently — real wall-clock win
SYNTHESIZE	Splunk hosted model fuses operational + code context
JUDGE	groundedness scored 0–1 · shown with every answer

▼

Grounded answer — specific log + specific file/method + fix

WHY IT WINS

Judged on four criteria

Technological Implementation

Two real MCP servers composed, hosted-model synthesis, parallel execution, groundedness scoring. Runs in ~1s.

Quality of the Idea

Dual-MCP fusion — the most architecturally sophisticated way to use Splunk's MCP Server as an interop layer.

Potential Impact

Every developer investigates incidents. 13 min → 45 s, with a groundedness guarantee on every answer.

Design

Dual-source investigation UI — Splunk green + SigMap teal, evidence panels, groundedness badge.

Squarely on the **Platform & DevEx** mandate: *simplify Splunk interactions.*

WHY NEITHER SOURCE ALONE IS ENOUGH

Splunk + code, or you're guessing

Splunk alone

"NullPointerException at line 47" — *which file?* You still open the IDE.

SigMap alone

"JwtTokenProvider.validateToken()" — *which error, when?* You still open Splunk.

Together

Specific file + specific error + root cause + fix. One grounded answer, scored.

In the demo incident, **neither source alone was sufficient** — Splunk had the line, SigMap had the method; only together did they pinpoint the bug.

TWO CONTEXTS. ONE ANSWER.

DualContext

Splunk MCP + SigMap MCP, fused by a Splunk hosted model. Grounded, every time.

Splunk MCP · SigMap MCP · Hosted Models · Groundedness · SPL

`github.com/manojmallick/dualcontext`